

손찬양 경력기술서

Java · Spring Boot 백엔드 개발자

github.com/cyson21 cyson21.github.io/portfolio/

API 개발·운영과 데이터 정합성

2021년부터 백엔드 개발·운영을 담당해 왔으며, Java·Spring Boot를 주력으로 사용합니다.

B2B 리테일 교육 플랫폼과 공공·실시간 데이터 서비스의 API를 개발·운영했습니다. 상태 변경, 다중 서버의 배치 중복 실행, 구매 API 락 경합과 데이터 식별자 오류를 분석하고 서버 로직을 수정했습니다.

회사별 담당 업무

이엠캐스트(주)

2024.03 – 현재 · 백엔드 개발자 · 주임

4인 개발팀에서 20개 이상의 기업 고객 서비스를 제공하는 B2B 리테일 교육 플랫폼의 백엔드 개발·운영을 담당했습니다.

- **교육 플랫폼 API 개발·운영**

Java·Spring Boot로 B2B 리테일 교육 플랫폼의 REST API를 설계·개발·운영했습니다. JPA·QueryDSL·MySQL로 조회·저장을 구현하고 관리자 기능 개편을 지원했습니다.

- **버전 전환·데이터 접근 계층 개선**

플랫폼 2.0 → 3.0 전환의 영향 범위를 점검하고 기존 기능의 회귀를 검증했습니다. 레거시 코드와 API·DB 구조, JPA·QueryDSL 조회·저장 로직을 정리했습니다.

- **운영 오류 분석·수정·회귀 검증**

운영 장애와 데이터 오류를 재현해 원인을 추적하고 API·DB 로직을 수정했습니다. 저장 시점 상태 검증, 분산락 기반 배치 제어와 구매 API 트랜잭션 분리, 복합 식별자 처리와 회귀 테스트를 다뤘습니다.

- **AWS 인프라·CI/CD·배포 운영**

AWS EC2·RDS·Lambda·CloudWatch·WAF 기반 배포·모니터링·운영에 참여했습니다. Docker 배포·전환 이슈에 대응하고 CI/CD 파이프라인 안정화와 코드리뷰 기반 배포 품질 관리에 참여했습니다.

주식회사 화이트스캔

2021.07 – 2024.03 · 백엔드·데이터 개발자 · 연구원

공공·실시간 데이터 기반 서비스의 백엔드와 데이터 처리, 배포·운영을 담당했습니다.

- **도시데이터 수집·가공·조회 API**

서울 실시간 도시데이터 Open API를 수집·가공하는 파이프라인과 조회 API를 구현했습니다.

- **데이터 모델·백엔드 개발**

서비스 요구사항에 맞춰 MySQL·MongoDB 스키마와 REST API를 설계하고, Spring Boot·Django·FastAPI로 데이터 조회·저장 기능을 구현했습니다.

- **예측 결과 연동·Docker 운영**

시계열 예측 결과를 서비스 지표와 기능에 연동하고 관련 서비스를 Docker 컨테이너로 배포·운영했습니다.

대표 실무 문제 해결

이앰캐스트 · B2B 리테일 교육 플랫폼

저장 시점의 상태 변경으로 발생하는 데이터 오류 방지

문제 설정 변경 화면을 연 뒤 관련 작업이 시작되면, 이전 상태를 기준으로 저장하면서 진행 중인 작업에 서로 다른 처리 기준이 적용될 수 있었습니다.

원인 확인 화면 진입 시점의 상태를 저장할 때도 유효한 값으로 취급했습니다. 변경 허용 여부와 후속 데이터 갱신도 같은 상태를 기준으로 판단하지 않았습니다.

판단 화면을 연 뒤에도 상태가 바뀔 수 있으므로 변경 허용 여부는 최종 저장 요청에서 판단해야 했습니다. 변경 차단과 후속 갱신이 서로 다른 기준을 사용하지 않도록 서버의 상태 조회 기준을 통일했습니다.

수정 저장 직전 서버에서 현재 상태를 다시 확인하도록 수정했습니다. 화면 진입 후 작업이 시작됐다면 변경을 차단하고, 후속 데이터 갱신 기준도 같은 상태 조회로 통일했습니다.

검증 근거 정상 변경, 저장 전 상태 전이, 기존 이력 존재, 필수 입력 누락을 다루는 단위 테스트와 당시 결과 기록이 있습니다.

[관련 실무 사례 · 웹 포트폴리오](#)

구매 API의 락 경합과 트랜잭션 경계 조정

문제 동시 구매 요청에서 주문 저장 중 DB 데드락이 발생했습니다. 결제 처리를 별도 트랜잭션으로 옮기자 앞 단계의 미커밋 상태를 읽는 문제도 드러났습니다.

원인 확인 주문 저장·컬렉션 갱신에서 락이 경합했습니다. 트랜잭션 분리 후에는 결제 처리가 앞 단계의 커밋보다 먼저 실행돼 필요한 요청 상태를 조회하지 못했습니다.

판단 동시 실행 제어와 트랜잭션 간 데이터 가시성을 별개로 다뤄야 했습니다. 락 적용 범위와 커밋 경계를 각각 조정하고, 모든 실패를 반복하지 않도록 재시도는 락 관련 예외에 한정했습니다.

수정 주문 단위 분산락과 별도 트랜잭션, 락 오류의 제한적 재시도를 적용했습니다. 결제 요청 상태를 먼저 커밋하도록 호출 경계를 조정하고, 재처리 시 결제 완료 상태를 확인했습니다.

검증 근거 동시 요청, 락 예외의 재시도, 결제 완료 주문의 재처리를 다루는 테스트 정의가 있습니다. 초기 QA에서 드러난 요청 상태 조회 문제와 이를 수정한 후속 기록을 함께 남겼습니다.

[관련 실무 사례 · 웹 포트폴리오](#)

대표 실무 문제 해결

이앰캐스트 · B2B 리테일 교육 플랫폼

배치 중복 실행 제어와 실패·실행 누락 모니터링

문제 서버마다 스케줄러가 실행돼 같은 배치가 중복 처리됐습니다. 고객사별로 예정된 배치의 실패와 실행 누락도 구분해야 했습니다.

원인 확인 서버별 스케줄러 사이에 실행 주체를 정하는 제어가 없었습니다. 실제 실행 기록만 보면 예정된 작업의 발행 누락과 발행 후 결과 미수신도 구분하기 어려웠습니다.

판단 중복 실행을 제어하는 락과 실패·누락을 발견하는 모니터링은 역할이 달랐습니다. 실행된 작업의 기록뿐 아니라 예정된 고객사·배치 목록을 대조해야 아예 실행되지 않은 작업도 찾을 수 있었습니다.

수정 ShedLock-Redis 분산락을 적용했습니다. 별도 개선으로 고객사·배치별 상태를 집계하고, 예정 시각 1시간 후 실패·발행 누락·결과 미수신을 구분해 Slack으로 알리도록 구현했습니다.

검증 근거 조치 후 서버 로그와 DB 기록으로 배치 실행 결과를 확인했습니다. 모니터링에는 고객사·배치 식별, 성공·실패 상태, 처리 건수와 실행 시간을 기록했습니다. 예정 목록 대조와 시간대별 집계·Slack 알림은 소스 및 관련 테스트 정의에 남아 있습니다.

[관련 실무 사례 · 웹 포트폴리오](#)

복합 식별자 누락으로 발생한 삭제·페이지 집계 오류 수정

문제 같은 사용자의 여러 그룹 데이터를 사용자 ID만으로 구분해 삭제 대상이 누락되고, 조회 제외 조건과 페이지 집계도 실제 행 수와 어긋났습니다.

원인 확인 사용자 ID만 비교하는 동등성·제외 조건이 서로 다른 그룹의 행을 같은 대상으로 취급했습니다. 조회 후 필터링과 별도 합계 계산은 페이지 내용과 총건수에도 다른 기준을 적용했습니다.

판단 삭제 대상과 컬렉션에 남길 행을 같은 식별 기준으로 구분해야 했습니다. 페이지 내용과 총건수도 동일한 필터 조건이 필요해, 조회 후 제외하던 처리를 DB 쿼리 단계로 옮겼습니다.

수정 사용자 ID와 그룹 유형의 복합 식별자를 조회·삭제·컬렉션 동등성에 적용했습니다. 제외 조건을 DB 쿼리로 옮기고 페이지 집계를 맞췄습니다.

검증 근거 같은 사용자의 서로 다른 그룹을 Set에서 보존하는 경우, 선택한 조합만 삭제하는 경우, 일부 그룹 제외와 페이지 경계를 다루는 테스트 및 당시 검증 기록이 있습니다.

[관련 실무 사례 · 웹 포트폴리오](#)

개인 프로젝트의 코드·테스트 근거는 cyson21.github.io/projects/에서 확인할 수 있습니다.